Machine Learning

Basic Algorithms

Concept · Worked example · Working code



PART ONE

Supervised learning

Learns from labeled input–output pairs. You give it the right answers, it learns the mapping.

1 Linear regression

Fits a straight line to predict a continuous number.

HOW IT WORKS

- Finds the line score = m x hours + b that sits closest to all points.
- "Closest" = smallest sum of squared vertical gaps (residuals).
- Squaring punishes big misses more than small ones.
- Use it when the output is a number: price, score, temperature.

EXAMPLE

Hours studied to exam score

1 hr -> 50 4 hrs -> 70

2 hrs -> 55 5 hrs -> 85

3 hrs -> 65

Learned line: score = 8.5 x hours + 42

Predict 6 hrs -> 93

1 Linear regression — code

```
import numpy as np, matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# dataset: hours studied -> exam score
X = np.array([[1], [2], [3], [4], [5]])     # 2D: rows = samples
y = np.array([50, 55, 65, 70, 85])          # 1D: one target each

model = LinearRegression()
model.fit(X, y)                             # learns m and b

print(model.coef_[0], model.intercept_)     # 8.5 , 42
print(model.predict([[6]]))                 # -> 93

xs = np.linspace(0.5, 6.5, 100).reshape(-1, 1)
plt.scatter(X, y, color='tomato')           # the data
plt.plot(xs, model.predict(xs), color='steelblue')   # the line
plt.show()
```

Key idea: .fit() finds the best line, .predict() uses it. reshape(-1,1) makes the 2D shape sklearn needs.

2 Logistic regression

Despite the name, this is classification — it predicts a class, not a number.

HOW IT WORKS

- Pushes a linear score through a sigmoid to get a probability 0–1.
- Applies a threshold (usually 0.5) to pick the class.
- Output is an S-curve, not a straight line.
- Use it for yes/no questions: pass/fail, spam/not spam, churn/stay.

EXAMPLE

Will the student pass?

1.5 hrs -> P(pass) = 0.08 -> FAIL

3.5 hrs -> P(pass) = 0.52 -> PASS

5.0 hrs -> P(pass) = 0.91 -> PASS

Threshold at 0.5 splits the two classes.

2 Logistic regression — code

```
from sklearn.linear_model import LogisticRegression

# dataset: hours studied -> 0 = fail, 1 = pass
X = np.array([[1], [1.5], [2], [3], [4], [5], [5.5]])
y = np.array([0, 0, 0, 0, 1, 1, 1])          # categorical target

model = LogisticRegression()
model.fit(X, y)

for hrs in [1.5, 3.5, 5]:
    p = model.predict_proba([[hrs]])[0][1]  # col 1 = P(pass)
    print(hrs, round(p, 2), 'PASS' if p > 0.5 else 'FAIL')

xs = np.linspace(0, 7, 300).reshape(-1, 1)
plt.plot(xs, model.predict_proba(xs)[:, 1])  # the S-curve
plt.axhline(0.5, ls='--', color='gray')      # threshold
plt.show()
```

predict_proba gives probabilities; predict gives the hard label. Column 1 is the positive class.

3 Decision tree

A flowchart of yes/no questions you can read top to bottom.

HOW IT WORKS

- At each step picks the question that best separates the classes.
- "Best" is measured by Gini impurity or information gain.
- Fully interpretable — you can explain every decision.
- Limit depth, or it memorises the training data (overfitting).

EXAMPLE

Loan approval

```
Income > $50k ?
  No   ->  REJECT
  Yes  ->  Credit score > 700 ?
             No   ->  REJECT
             Yes  ->  APPROVE

Applicant: $75k, score 780  ->  APPROVE
```

3 Decision tree — code

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

# dataset: [income_k, credit_score] -> 1 = approve
X = [[30, 650], [40, 720], [45, 690], [60, 750], [70, 800],
     [55, 640], [80, 710], [35, 800], [90, 660], [65, 730]]
y = [0, 0, 0, 1, 1, 0, 1, 0, 0, 1]

model = DecisionTreeClassifier(max_depth=2, random_state=0)
model.fit(X, y)                # max_depth stops overfitting

print(model.predict([[75, 780]]))          # -> [1] APPROVE

plt.figure(figsize=(10, 6))
plot_tree(model, feature_names=['income_k', 'credit_score'],
          class_names=['Reject', 'Approve'], filled=True)
plt.show()
```

plot_tree draws the actual learned tree — the clearest algorithm to show students, because the logic is visible.

4 K-nearest neighbours

No training at all — it just remembers the data and votes.

HOW IT WORKS

- Stores every training point; does the work at prediction time.
- For a new point, finds the k closest points by distance.
- Takes a majority vote among those k neighbours.
- k is the one parameter to tune: small k = noisy, large k = blurry.

EXAMPLE

Apple or orange?

Features: weight, redness

New fruit: [2.7, 1.5]

3 nearest neighbours:
apple, apple, apple

Vote -> APPLE

4 K-nearest neighbours — code

```
from sklearn.neighbors import KNeighborsClassifier

# dataset: [weight, redness]
X = np.array([[1.8, 1.2], [2.2, 1.6], [2.6, 1.0], [1.5, 1.8],
              [4.0, 2.5], [4.5, 2.2], [4.8, 2.7], [4.2, 2.0]])
y = np.array(['apple'] * 4 + ['orange'] * 4)   # string labels are fine

model = KNeighborsClassifier(n_neighbors=3)    # k = 3
model.fit(X, y)                # just stores the data

new = np.array([[2.7, 1.5]])
print(model.predict(new))                      # -> ['apple']

dist, idx = model.kneighbors(new)   # who voted, and how far
print([y[i] for i in idx[0]])       # the 3 neighbour labels
```

KNN's .fit() does no learning — all computation happens at predict time. That makes it slow on big datasets.

5

Support vector machine

Draws the boundary with the widest possible gap between classes.

HOW IT WORKS

- Finds the separating line that maximises the margin.
- Only the closest points — the support vectors — define it.
- All other training points could be deleted without changing it.
- The kernel trick bends the boundary for non-separable data.

EXAMPLE

Spam vs business email

Features: spammy words, business words

Spam: [5,0] [6,1] [4,0] [7,1]
Ham: [0,5] [1,6] [0,4] [1,7]

Boundary sits exactly midway,
held in place by the support vectors.

5 Support vector machine — code

```
from sklearn.svm import SVC

# dataset: [spammy_words, business_words] -> 1 = spam
X = np.array([[5, 0], [6, 1], [4, 0], [7, 1],
              [0, 5], [1, 6], [0, 4], [1, 7]])
y = np.array([1, 1, 1, 1, 0, 0, 0, 0])

model = SVC(kernel='linear')   # 'rbf' curves the boundary
model.fit(X, y)

print(model.support_vectors_)  # the points that define it

xx, yy = np.meshgrid(np.linspace(-1, 8, 200),
                     np.linspace(-1, 8, 200))
Z = model.decision_function(np.c_[xx.ravel(), yy.ravel()])
plt.contour(xx, yy, Z.reshape(xx.shape), levels=[-1, 0, 1])
plt.show()                     # boundary + both margins
```

The meshgrid + decision_function + contour pattern draws ANY 2D decision boundary — reuse it for other classifiers.

6 Naive Bayes

Multiplies word probabilities together using Bayes' theorem.

HOW IT WORKS

- Learns P(word | spam) for every word from training data.
- Multiplies them to score a new message.
- "Naive" = assumes words are independent of each other.
- That assumption is false, yet it works very well on text.

EXAMPLE

Spam detection

P('free' | spam) = 0.8
P('winner' | spam) = 0.9
P('click' | spam) = 0.6

Email: "free winner click"

Combined -> classified SPAM

6 Naive Bayes — code

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

emails = ['free winner click now', 'free money winner',
          'click free prize',      'meeting tomorrow report',
          'project deadline monday', 'lunch meeting notes']
labels = ['spam', 'spam', 'spam', 'ham', 'ham', 'ham']

vec = CountVectorizer()         # text -> word counts
X = vec.fit_transform(emails)   # builds vocab + transforms

model = MultinomialNB()         # NB variant for count data
model.fit(X, labels)

test = vec.transform(['free winner click'])
print(model.predict_proba(test))    # P(ham), P(spam)
print(model.predict(test))          # -> ['spam']
```

CountVectorizer turns text into numbers first — every text model needs this step. Use transform (not fit_transform) on test data.

PART TWO

Unsupervised learning

No labels at all. The algorithm finds structure in the data on its own.

7

K-means clustering

Groups data into k clusters with no labels given.

HOW IT WORKS

- Pick k. Place k centroids at random starting positions.
- Assign every point to its nearest centroid.
- Move each centroid to the average of its assigned points.
- Repeat until the centroids stop moving.

EXAMPLE

Customer segments

Features: purchases/month, avg spend

Cluster 0: low spenders (2.5, $32)
Cluster 1: frequent buyers (13.5, $82)
Cluster 2: big spenders (4, $312)

Found without being told the segments.

7 K-means clustering — code

```
from sklearn.cluster import KMeans

# dataset: [purchases/month, avg spend $] - NO labels
X = np.array([[2, 30],  [3, 35],  [2, 25],  [3, 40],
              [12, 80], [14, 90], [13, 75], [15, 85],
              [4, 300], [5, 350], [3, 280], [4, 320]])

model = KMeans(n_clusters=3, random_state=0, n_init=10)
labels = model.fit_predict(X)   # fit + assign in one call

print(labels)                   # cluster id per customer
print(model.cluster_centers_)   # the 3 centroids

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='Set2')
plt.scatter(*model.cluster_centers_.T, marker='X', s=300)
plt.show()
```

Note there is no y anywhere — that is what makes it unsupervised. fit_predict trains and assigns in one step.

8 PCA

Squeezes many features into a few, keeping most of the information.

HOW IT WORKS

- Finds new axes pointing along the directions of greatest variance.
- Projects the data onto just the top few of those axes.
- Always scale your features first, or big units dominate.
- Used to speed up models and to make data visualisable in 2D.

EXAMPLE

House features

5 features per house:
sqft, bedrooms, age, distance, garage

Compressed to 2 components

PC1 keeps 71% of the variance
PC2 keeps 21% -> 92% total

8 PCA — code

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# [sqft, bedrooms, age, dist_to_school, garage]
X = np.array([[1500, 3, 20, 2.0, 1], [2400, 4, 5, 1.5, 2],
              [1100, 2, 35, 3.0, 1], [3000, 5, 2, 0.8, 3],
              [1800, 3, 15, 2.2, 2], [2200, 4, 10, 1.2, 2]])

X = StandardScaler().fit_transform(X)   # ALWAYS scale first

pca = PCA(n_components=2)               # 5 features -> 2
X2 = pca.fit_transform(X)

print(pca.explained_variance_ratio_)    # info kept per axis
print(pca.explained_variance_ratio_.sum())

plt.scatter(X2[:, 0], X2[:, 1])
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.show()
```

Skipping StandardScaler is the classic PCA mistake — sqft in thousands would swamp bedrooms in single digits.

Which one do I use?

Predict a number	**Linear regression**
Predict a yes/no class	**Logistic regression, SVM, Naive Bayes**
Need to explain the decision	**Decision tree**
Small data, no training time	**K-nearest neighbours**
Working with text	**Naive Bayes**
No labels — find groups	**K-means clustering**
Too many features	**PCA**